Architecture.md v1.1

Chapter 11 — Configuration Management & Environment Architecture

This chapter defines how AI OS manages configuration, environment variables, feature flags, AI providers, and runtime settings.

Configuration is a **Core Service** and shall remain centralized.

11.1 Purpose

The Configuration Service shall:

- Store runtime settings.
- Manage AI providers.
- Manage API credentials.
- Control feature flags.
- Configure environments.
- Provide a single source of configuration.

Configuration is read by all components but managed centrally.

11.2 Design Philosophy

Configuration follows five principles.

Centralized

All configuration originates from one Configuration Service.

Environment-Based

Development, testing, and production environments remain independent.

Secure

Secrets never appear in source code.

Versioned

Configuration changes are traceable.

Replaceable

Changing providers or models must require only configuration updates.

11.3 Configuration Sources

AI OS uses three configuration layers.

Environment Variables

Stored in:

`.env`

Contains:

- API Keys
- Tokens
- Passwords
- Database credentials

System Configuration

Stored in:

`config.yaml`

Contains:

- Feature flags
- Default AI models
- Logging levels
- Paths

- Timeouts

Runtime Configuration

Managed by the Configuration Service.

Contains:

- Active AI provider
- User preferences
- Current operating mode

Runtime changes do not modify source code.

11.4 Environment Types

Three official environments are defined.

Development

Purpose:

Local development.

Characteristics:

- Debug enabled
- Verbose logging
- Mock services allowed

Testing

Purpose:

Automated testing.

Characteristics:

- Temporary database
- Mock APIs
- Deterministic behavior

Production

Purpose:

Daily usage.

Characteristics:

- Debug disabled
- Optimized logging
- Secure secrets
- Performance optimized

11.5 AI Provider Configuration

The AI provider is configurable.

Supported providers may include:

- OpenAI
- Google Gemini
- Anthropic Claude
- Local Models
- Future Providers

The architecture never depends on one provider.

Providers are selected through configuration.

11.6 Model Configuration

Each subsystem may define a preferred model.

Example:

Jarvis

↓

Reasoning Model

Friday

↓

Fast Execution Model

Research Agent

↓

Large Context Model

Configuration determines model selection.

11.7 Feature Flags

Feature flags enable or disable capabilities.

Examples:

```text id="3sjj2h" voice_enabled: true

browser_agent: true

trading_agent: false

debug_dashboard: false
```

```
Feature flags require no code modifications.

---

# 11.8 System Paths

Centralized path configuration.

Examples:

- Logs
- Database
- Uploads
- Backups
- Temporary files

Components never hard-code paths.
```

```
---

# 11.9 Timeout Configuration

Configurable values include:

- Router timeout
- Agent timeout
- Database timeout
- Workflow timeout
- API timeout

Defaults are defined centrally.

---

# 11.10 Retry Configuration

Retry behavior is configurable.

Parameters include:

- Maximum retries
- Retry interval
- Retry strategy

Worker agents do not define their own retry policies.

---

# 11.11 Agent Configuration

Every Worker Agent owns a local configuration section.

Examples:

Analytics Agent:

- Report depth
- Default metrics
- Cache duration

Browser Agent:

- User agent
- Download directory
- Request timeout
```

```
Global settings always override local conflicts.

---

# 11.12 Configuration Loading

Startup sequence:

```text id="uzd89d"
Load .env
      │
      ▼
Load config.yaml
      │
      ▼
Validate Configuration
      │
      ▼
Initialize Services
      │
      ▼
System Ready
```

Startup fails if required configuration is missing.

11.13 Configuration Validation

The Configuration Service validates:

- Required fields
- Data types
- File paths
- API credentials
- Environment consistency

Invalid configuration prevents system startup.

```

11.14 Configuration Updates

Configuration changes require:

- Validation
- Version increment
- Audit log entry

Critical settings may require user approval.

11.15 Default Values

Every configurable option should provide a safe default where practical.

Examples:

- Logging Level
- Retry Count
- Timeout Values

Secrets must never have default values.

11.16 Future Expansion

Future versions may support:

- Remote configuration
- Cloud synchronization
- User profiles
- Dynamic feature rollout
- Plugin configuration
- Multi-device settings

The Configuration Service remains the single entry point.

11.17 Design Rules

The Configuration System must satisfy:

- Centralized management.
- Environment separation.

- Secure secret storage.
- Provider independence.
- Version-controlled changes.
- Validated startup.
- No hard-coded credentials.

No component may maintain an independent configuration system outside this architecture.

End of Chapter 11

Next Chapter: Chapter 12 — Plugin Architecture & Agent Registration System